Lab Assignment 1

Submitted by Musa Sajjad SP24-BDS-039

Submitted to Sir Zaeem Akbar

Submitted on 29th September 2026

```
# Download necessary models for NLTK and spaCy
import nltk
nltk.download('punkt')
nltk.download('punkt_tab')

!python -m spacy download en_core_web_sm
```

```
[nltk_data] Downloading package punkt to /root/nltk_data...
[nltk_data]   Unzipping tokenizers/punkt.zip.
[nltk_data] Downloading package punkt_tab to /root/nltk_data...
[nltk_data]   Unzipping tokenizers/punkt_tab.zip.
Collecting en-core-web-sm==3.8.0
  Downloading https://github.com/explosion/spacy-models/releases/download
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 12.8/12.8 MB 55.3 MB
✔ Download and installation successful
You can now load the package via spacy.load('en_core_web_sm')
⚠ Restart to reload dependencies
If you are in a Jupyter or Colab notebook, you may need to restart Python
order to load all the package's dependencies. You can do this by selectin
'Restart kernel' or 'Restart runtime' option.
```

Task 1: Regex Fundamentals

Extracting digits, capitalized words, 4-digit years, and words of specific lengths using regular expressions.

```
import re

sample_text = "In 2024, Alice and Bob bought 15 apples and 300 oranges from Mark

# a) Extract all digits
digits = re.findall(r'\d+', sample_text)
print("a) Digits:", digits)

# b) Extract words starting with a capital letter
capital_words = re.findall(r'\b[A-Z][a-zA-Z]*\b', sample_text)
print("b) Capitalized words:", capital_words)
```

```
# c) Extract exactly 4-digit sequences (e.g., years)
four_digits = re.findall(r'\b\d{4}\b', sample_text)
print("c) 4-digit sequences:", four_digits)

# d) Extract words between 3 to 6 characters long
three_to_six = re.findall(r'\b[a-zA-Z]{3,6}\b', sample_text)
print("d) Words with 3-6 letters:", three_to_six)
```

```
a) Digits: ['2024', '15', '300']
b) Capitalized words: ['In', 'Alice', 'Bob', 'Market']
c) 4-digit sequences: ['2024']
d) Words with 3-6 letters: ['Alice', 'and', 'Bob', 'bought', 'apples', 'a
```

Task 2: Pattern-Based Extraction (Real World Data)

Extracting emails, phone numbers (03XX-XXXXXXX), dates (DD/MM/YYYY), and percentages.

```
import re

paragraph = """
Hello, please contact support at help@company.com or sales@store.pk.
Call our hotline at 0300-1234567 or 0321-7654321.
The deadline is 15/08/2024, and the discount offers are 20% and 12.5%.
"""

# a) Extract email addresses
emails = re.findall(r'[\w\.-]+@[\w\.-]+\.\w+', paragraph)
print("a) Emails:", emails)

# b) Extract phone numbers (format: 03XX-XXXXXXX)
phones = re.findall(r'03\d{2}-\d{7}', paragraph)
print("b) Phone Numbers:", phones)

# c) Extract dates in DD/MM/YYYY format
dates = re.findall(r'\b\d{2}/\d{2}/\d{4}\b', paragraph)
print("c) Dates:", dates)

# d) Extract percentage values (e.g., 45%, 12.5%)
percentages = re.findall(r'\b\d+(?:\.\d+)?%', paragraph)
print("d) Percentages:", percentages)
```

```
a) Emails: ['help@company.com', 'sales@store.pk']
b) Phone Numbers: ['0300-1234567', '0321-7654321']
c) Dates: ['15/08/2024']
d) Percentages: ['20%', '12.5%']
```

Task 3: Word Tokenization (NLTK vs spaCy)

Comparing tokenization behavior and total token counts between NLTK and a blank spaCy pipeline.

```
import nltk
from nltk.tokenize import word_tokenize
import spacy

sample_para = "It's a sunny day, but I don't want to go out. The bird's wings ar

# a) Tokenize using NLTK
nltk_tokens = word_tokenize(sample_para)
print("a) NLTK Tokens:\n", nltk_tokens)

# b) Tokenize using blank spaCy pipeline
nlp_blank = spacy.blank("en")
spacy_tokens = [token.text for token in nlp_blank(sample_para)]
print("\nb) spaCy Tokens:\n", spacy_tokens)

# c) Differences noticed:
# Difference 1: Handling contractions. NLTK splits "don't" into ['do', "n't"], w
# Difference 2: Handling punctuation with words. NLTK treats punctuation attache

# d) Count total tokens produced
print("\nTotal NLTK Tokens:", len(nltk_tokens))
print("Total spaCy Tokens:", len(spacy_tokens))
```

```
a) NLTK Tokens:
 ['It', "'s", 'a', 'sunny', 'day', ',', 'but', 'I', 'do', "n't", 'want',

b) spaCy Tokens:
 ['It', "'s", 'a', 'sunny', 'day', ',', 'but', 'I', 'do', "n't", 'want',

Total NLTK Tokens: 29
Total spaCy Tokens: 29
```

Task 4: Token Attributes (Categorizing Tokens)

Inspecting built-in token attributes: is_alpha, like_num, like_email, and is_currency.

```
import spacy

nlp = spacy.blank("en")

sentence = "Send $50 or Rs. 500 to user@example.com for 2 tickets today."
doc = nlp(sentence)

# b & c) Loop through each token and print is_alpha, like_num, like_email
print(f"{'Token':<18} | {'is_alpha':<10} | {'like_num':<10} | {'like_email':<10}
print("-" * 55)

for token in doc:
    print(f"{token.text:<18} | {str(token.is_alpha):<10} | {str(token.like_num):
```

```
# d) Identify and list tokens where is_currency is True
currency_tokens = [token.text for token in doc if token.is_currency]
print("\nTokens where is_currency is True:", currency_tokens)
```

```
Token               | is_alpha   | like_num   | like_email
-----------------------------------------------------------
Send                | True       | False      | False
$                   | False      | False      | False
50                  | False      | True       | False
or                  | True       | False      | False
Rs                  | True       | False      | False
.                   | False      | False      | False
500                 | False      | True       | False
to                  | True       | False      | False
user@example.com    | False      | False      | True
for                 | True       | False      | False
2                   | False      | True       | False
tickets             | True       | False      | False
today               | True       | False      | False
.                   | False      | False      | False

Tokens where is_currency is True: ['$']
```

Task 5: Sentence Segmentation (Handling Tricky Cases)

Splitting text containing middle-sentence abbreviations using a trained spaCy pipeline (`en_core_web_sm`).

```
import spacy

# a) Load trained spaCy pipeline
nlp = spacy.load("en_core_web_sm")

text = "Dr. Smith arrived at the clinic early this morning. He met Mr. John to d
doc = nlp(text)

# b) Split the paragraph into individual sentences
sentences = list(doc.sents)
print("Detected Sentences:")
for i, sent in enumerate(sentences, 1):
    print(f"{i}. {sent.text}")

# c) Print the total number of sentences detected
print("\nTotal sentences detected:", len(sentences))

# d) Observation:
# The abbreviation "Dr." and "Mr." did NOT cause an incorrect sentence break bec
```

```
Detected Sentences:
1. Dr. Smith arrived at the clinic early this morning.
2. He met Mr. John to discuss the test results.

Total sentences detected: 2
```